AMALITECH × AMERICAN INDUSTRIAL PARTNERS · ENGINEERING HACKATHON

Ten portfolio problems, two days, one orchestrator

Every brief below is a real problem inside an AIP portfolio company, cut down to something four engineers can prototype in two days on AWS. None of them is a chatbot.

Format teams of 3–4 · 2 working days **Stack** Amazon Bedrock · AgentCore · Strands

Pick one brief per team, first come **Judged on** §5

01

The brief

AIP buys mid-market industrial companies — explosives, drilling, industrial services, lubricants distribution — and needs them worth more in five years than they were at close. The tech problems that stand in the way are unglamorous, specific, and remarkably consistent from one portfolio company to the next.

That consistency is the opportunity. A prototype that works on one portfolio company's problem is a template for the next four. This is why we are running a hackathon rather than writing a roadmap: we want ten shots at the question *which of these is worth building properly*.

The four rules

1. **A portfolio-company executive has to want it.** Not the CIO — the COO, the CFO, the VP of operations. If you cannot name the person whose week gets better, you have picked a technology, not a problem.
2. **It has to be agentic, and you have to be able to say why.** Multiple steps, a decision about what to do next, tools it chooses between. If a Lambda and a SQL query would do the job, use a Lambda and a SQL query and pick a different brief.
3. **Two days means two days.** Every brief names the cut line. Scope down to the demo, then down again.
4. **It ships documented.** §4 lists the files. A brilliant demo with no `AGENTS.md` scores below a modest one that has it, and that is deliberate.

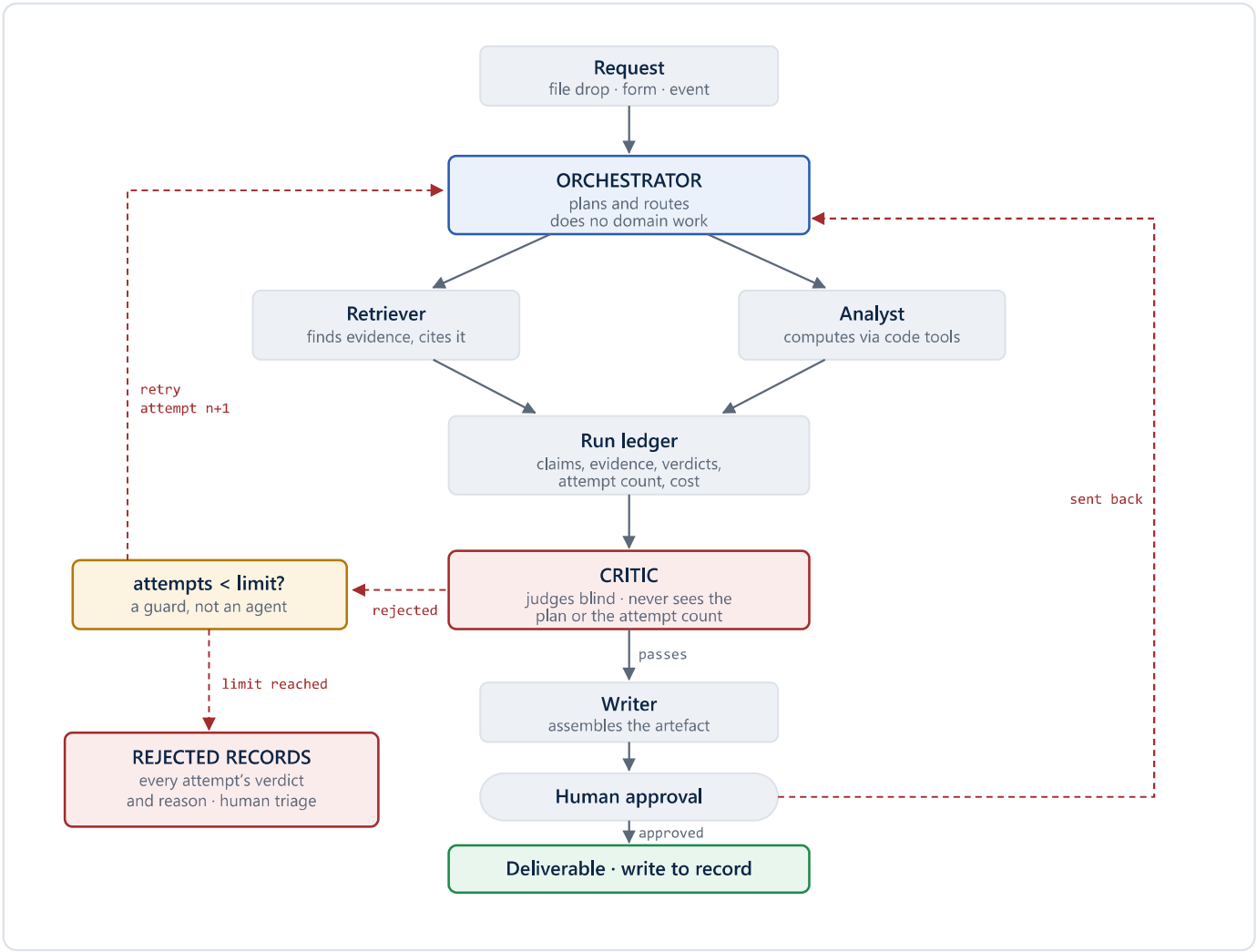
> ON USING REAL DATA
>
> Work in the hackathon sandbox account with synthetic or anonymised data only. No

> Work in the hackathon sandbox account with synthetic or anonymised data only. No portfolio-company production credentials, no real customer records, no financial statements. Several briefs list a data shape — generate data of that shape. A judge who sees a real client name in a prompt log stops the demo.

02

The orchestrator pattern

All ten briefs use the same shape. Learn it once. What changes between briefs is the specialists and the tools, never the skeleton.



The five roles, and what each is for

AGENT	SINGLE RESPONSIBILITY	WHAT IT MUST NEVER DO
`orchestrator`	Decide what happens next and in what order. Hold the plan. Stop when the goal is met or the budget is gone.	Answer a domain question itself. The moment it does, you have one big agent again.
`retriever`	Find evidence and return it with a citable identifier — document, page, row, timestamp.	Interpret. It returns what it found, not what it means.

`analyst`	Turn evidence into a claim. Arithmetic goes through a code tool, never through the model's head.	Invent a number that is not in the evidence it was given.
`critic`	Receive a claim plus its evidence — *without* the plan, the conversation, or how many attempts are left — and reject anything unsupported.	See the orchestrator's reasoning, or the retry counter. Its independence is the entire point.
`writer`	Assemble passed claims into the artefact a human asked for, carrying every citation through.	Add a claim of its own while writing. This is the most common way a hallucination reaches the deliverable.

THE CRITIC IS THE HIGHEST-VALUE AGENT YOU WILL BUILD

It is also the one teams cut first when they run late. Do not. Two days is enough to build something that produces confident output; the critic is what makes that output worth showing to someone who will act on it. A brief's demo lands when the critic *catches something on stage* — so keep a deliberately weak input in your demo set and let it get rejected in front of the judges.

THE REJECT LOOP IS THE ONE THAT WILL BITE YOU

Critic rejects → orchestrator re-plans → critic rejects has no floor in it. A claim the evidence will never support goes round until the account notices, and the failure is silent: no error, no crash, just a run that never finishes and a bill that does.

Bound it, and enforce the bound on the edge rather than inside the critic. Every rejection increments an attempt counter held in the run ledger against that claim — *not* carried in the message, where an agent can drop or reset it. The rejected path is then a guard, not a hop: under the limit it returns to the orchestrator to re-plan; at the limit the claim leaves the loop and is written to a **rejected-record store** with every attempt's verdict and reason, for a human queue that someone actually works.

Two retries is the right default for a 72-hour build. If three attempts cannot evidence a claim, the fault is almost never in the claim — it is upstream, in retrieval or in the data — and further attempts only spend money on the same dead end.

Why the counter is not the critic's job, even though that is the obvious place for it: a critic that knows it is on the last attempt has a reason to soften, and its independence is the only thing this whole pattern rests on. Let it judge blind every time and let the edge after it do the arithmetic.

The human *sent back* edge needs no limit — a person is the control, and that is the point of putting them there. Write each return to the same ledger anyway, so a draft that has been round four times is visible rather than merely slow.

Eight non-negotiables

1. **Every claim cites its source.** Claim without evidence identifier is a bug, not a soft output. Fail the run.
2. **Every loop has a bound and a terminal state.** A retry ceiling in the ledger, a guard on the rejected edge, and a rejected-record store at the end of it — as above. "It will converge" is not a design.
3. **No agent writes to a system of record.** Writes go behind a human approval step — Step Functions `waitForTaskToken` is four lines and it is the difference between a prototype we can show a client and one we cannot.
4. **Tools are code.** Anything deterministic — a join, a rate lookup, a date difference, a currency conversion — is a function the agent calls, not a paragraph it reasons through.
5. **State lives outside the agents.** A DynamoDB run ledger keyed by `run_id`, holding every claim, its evidence, the critic's verdict, **the attempt count** and the tokens spent. Runs become resumable, auditable and demoable after the fact — and the counter that stops the reject loop has somewhere tamper-proof to live.
6. **One source of truth per computed thing.** If two places decide the same number, they will eventually disagree, and the one on screen will be the wrong one. We have shipped that bug on this account four times; every occurrence was two components deciding independently.
7. **A budget ceiling per run, enforced in the orchestrator.** Fail closed with a partial result and a reason. An agent loop with no ceiling is how a hackathon account gets suspended on day two.
8. **Guardrails on every model call that touches client text.** Bedrock Guardrails, PII filter on. Free, one config, and it is the first thing a client's security lead asks about.

03

Ten use cases

Each brief names the portfolio reality it comes from, the agent roster, the AWS services, and — most importantly — the cut line: the smallest version that still demos.

UC-01 Week-zero data room triage

Diligence

> *On our last post-acquisition assessment the financial statements and ERP extracts did not arrive until week three or four. The engagement was sold as four weeks and ran five to six.*
>
> Delivery retrospective, September 2026

An assessment team's first three weeks go on orientation: what systems exist, where the money goes, which of these thirty PDFs matters. Meanwhile the clock is running on a fixed-

price engagement.

Build the agent that reads the data room the night it opens and hands the team a Day-1 hypothesis pack: the five things most likely to be wrong, each with the evidence behind it and the confidence it deserves.

AGENTS	`orchestrator` → `doc-classifier`, `schema-profiler` (ERP extracts), `spend-classifier` (invoices, contracts), `hypothesis-agent`, `critic`, `writer`
DATA SHAPE	~30 mixed PDFs and spreadsheets, 2 CSV table dumps with unlabelled columns, one folder of invoices
CUT LINE	Three document types, five hypotheses, no UI beyond a rendered Markdown pack. **Citations are not optional even in the cut version.**
DEMO MOMENT	Drop a folder in S3. Ninety seconds later, five findings with page-level citations — and one the critic rejected, shown with its reason.
WATCH FOR	Confidently summarising documents nobody asked about. The value is prioritisation, not coverage.

`S3` `Textract` `Bedrock KB` `Strands` `Step Functions` `DynamoDB`

UC-02 The interview glossary that learns

Knowledge capture

> *First time, you understand 20% of that. Three weeks later, you can actually make sense of 80 to 90% of what they said in the first meeting.*
>
> On transcribing assessment interviews

Between regional accents and a company's own industry vocabulary, the early interviews of any engagement are barely comprehensible to the team — and they are the interviews with the most senior people in them. By the time the team has the vocabulary, the access is gone.

Build a pipeline that transcribes each interview, *accumulates the client's vocabulary as it goes*, and retroactively re-reads earlier transcripts with everything learned since. Then map statements onto the ten IT maturity dimensions and flag where two interviewees contradict each other.

AGENTS	`orchestrator` → `glossary-builder` (writes to shared memory), `dimension-mapper`, `contradiction-finder`, `critic`, `writer`
DATA SHAPE	6–8 interview recordings, 20–40 min each, deliberately including jargon and at least one pair who disagree
CUT LINE	Three interviews, one dimension mapped properly rather than ten badly, and the re-read pass working. The re-read *is* the idea — do not cut it.
DEMO MOMENT	Interview one, read on arrival, next to interview one re-read after interview six. The diff is the product.
WATCH FOR	A glossary that is just a term list. It has to change how later passes interpret earlier

audio.

Transcribe | Bedrock | AgentCore Memory | Bedrock KB | S3

UC-03 Eighteen ERP systems, one map

Enterprise architecture

> *Eighteen ERP systems — three SAP ECC, five Business One, more inside the SAP family, and others from a different vendor entirely.*
>
> An AIP portfolio company's actual estate

Nobody in the company can draw it. The cost of an ERP programme is driven less by how many systems there are than by how many *different kinds* of system there are — three copies of one product is a fraction of the work of three different products — and no one can answer that question either.

Build the cartographer: connector agents read each system's schema and config export, a normaliser maps every one onto a canonical model, a duplicate-detector finds the same customer and material sitting in five places, and the orchestrator emits a landscape map, an integration inventory and a proposed migration wave order with the reasoning attached.

AGENTS	`orchestrator` → `schema-reader` (one per product), `canonical-mapper`, `entity-overlap-detector`, `wave-planner`, `critic`
DATA SHAPE	Six synthetic schema exports across three different ERP products, with overlapping customer and material masters
CUT LINE	Two products, four instances, and the overlap detection working. Drop the wave planner before you drop the overlap detection.
DEMO MOMENT	The generated landscape diagram beside the answer: *four vendors, seven products, eighteen instances* — and what each tier costs to learn.
WATCH FOR	A pretty diagram with no reconciliation underneath. The overlap is the insight; the diagram is the packaging.

Glue | Athena | Bedrock | Strands | OpenSearch Serverless

UC-04 Golden record, with its reasons

Data governance

> *Your master data governance is a mess.*
>
> Assessment finding, verbatim, more than once

The same customer exists seven times across seven instances, spelled differently each time. Off-the-shelf matching gets you 80% and then stalls, because the last 20% needs judgement — and a data steward will not accept a match they cannot interrogate.

Build a reconciliation crew where every proposed merge arrives with an argument. A

candidate-matcher does cheap blocking and embedding similarity, an adjudicator proposes the golden record *and writes down why*, a steward approves or rejects in a queue, and a rules-learner turns each human decision into a reusable rule so the queue shrinks as it is worked.

AGENTS	`orchestrator` → `candidate-matcher`, `adjudicator`, `rules-learner`, `critic` + **human steward queue**
DATA SHAPE	5,000 customer records across four instances with realistic mess: abbreviations, legal-entity suffixes, address variants, one genuine near-twin that must *not* merge
CUT LINE	One entity type, one merge dimension, a working approval queue. The rules-learner can be one rule type.
DEMO MOMENT	Approve four merges; watch the fifth get auto-proposed with higher confidence because of what it learned from the first four.
WATCH FOR	Merging the near-twin. Put one in your eval set and make sure you hold it apart.

`Bedrock embeddings` `OpenSearch Serverless` `DynamoDB` `Step Functions` `AgentCore Memory`

UC-05 Where the order-to-cash money leaks

Value chain

> *Here is a mapping of levers against pain points, and what we think your revenue and profit impact would be if you tackle this.*
>
> The core exhibit of a CEO board deck, built by hand

This is the deliverable that turns an IT finding into something a CEO acts on, and today it takes a senior consultant weeks. The inputs are not exotic: order, delivery and invoice events, which every ERP has.

Build the analyst that mines the real process from event data — not the documented one — finds the step where time and margin actually go, sizes the impact in currency, and maps each bottleneck to a fix with a cost estimate. Every number traceable to a row.

AGENTS	`orchestrator` → `event-loader`, `process-miner`, `impact-quantifier` (code tool, not prose), `lever-mapper`, `critic`, `writer`
DATA SHAPE	~200k order-to-cash events, 18 months, with a credit-check bottleneck and a regional delivery problem planted in it
CUT LINE	One value chain, three bottlenecks, impact sized on a stated assumption. State the assumption on the slide.
DEMO MOMENT	"Credit blocks add 4.1 days to 23% of orders; here are the eleven rows behind that number."
WATCH FOR	A model doing arithmetic. Every figure comes out of the code interpreter or it does not ship.

`Athena` `Bedrock Code Interpreter` `Strands` `QuickSight` `S3`

UC-06 Shadow IT, found by follow the money

Cost & FinOps

> *IT spend and savings potential, including shadow IT.*
>
> A standing line item in every assessment scope

Mid-market industrial companies routinely cannot say what they spend on IT, because a material share of it is bought on departmental cards and never passes through IT. The answer is reconstructable from data the finance team already has.

Build the detective: ingest accounts-payable lines, card transactions and SSO sign-in logs; classify every vendor; flag anything with spend but no IT owner, or an owner who left; and propose consolidations with a saving and an honest risk note beside each.

AGENTS	`orchestrator` → `vendor-normaliser`, `category-classifier`, `shadow-detector`, `savings-proposer`, `critic`
DATA SHAPE	~8k AP and card lines with inconsistent vendor naming, an SSO app list, an IT-owned application register that is deliberately incomplete
CUT LINE	AP lines plus the SSO list. Skip the cloud bill; the reconciliation between "paid for" and "signed into" is the whole trick.
DEMO MOMENT	Four tools paid for, two of them signed into by nobody for ninety days, one duplicating a tool IT already owns.
WATCH FOR	Claiming a saving without a risk note. A consolidation that breaks a workflow is not a saving.

`Textract` `Comprehend` `Athena` `Bedrock` `Guardrails`

UC-07 Forecast scoping in an afternoon

Machine learning

> *They just tell us how it is and give us some sample data, and we look at the data ourselves and see how complex it is.*
>
> How ML scoping works today

Scoping a forecasting engagement is a fixed amount of work regardless of how many things the client wants forecast — a standard set of interviews plus a read of sample data. It is fixed, repeatable, and almost entirely mechanical, which makes it the best automation target on the menu.

Build the scoping agent: profile each forecast target for quality, history depth, seasonality and intermittency; scout which external drivers are both useful and actually obtainable; recommend one-shot versus trained with a defensible reason; and produce the phase-one report a client pays for.

AGENTS	`orchestrator` → `data-profiler`, `feature-scout`, `model-selector`, `baseline-runner`, `critic`, `writer`

DATA SHAPE	Three targets of differing difficulty — one clean seasonal series, one intermittent and spiky, one with an obvious external driver
CUT LINE	Two targets, a naive baseline actually run, one external driver checked for real availability. A recommendation with no baseline behind it is an opinion.
DEMO MOMENT	Two targets, two different recommendations, and the specific property of each dataset that drove the difference.
WATCH FOR	Recommending a foundation model for everything. The interesting output is *"this one does not need training and here is the evidence"*.

`Bedrock` `Code Interpreter` `SageMaker` `Step Functions` `S3`

UC-08 Incident learnings that travel between sites

Safety & compliance

> *Is somebody doing the pen testing for you? Have there been remediations? Or are you stuck actioning them?*
>
> The same question, asked of safety findings

An explosives manufacturer and an industrial services contractor both carry heavy regulatory load, report incidents as free text, and have no way to see that the same root cause has now appeared at four sites. The corrective action is written locally and the learning stays local.

Build the copilot: normalise incident narratives, cluster by cause across sites, retrieve the applicable regulation from a corpus the team supplies, assess reportability, and draft a corrective action that a named human signs. Nothing is ever filed automatically.

AGENTS	`orchestrator` → `narrative-normaliser`, `cause-clusterer`, `regulation-retriever`, `capa-drafter`, `critic` + **mandatory human sign-off**
DATA SHAPE	400 synthetic incident narratives across six sites, with one root cause recurring in three of them under different wording, plus a small public regulation corpus
CUT LINE	Clustering plus the reportability question. The corrective-action drafter can be a template.
DEMO MOMENT	"This has happened three times, at three sites, described three different ways" — with the three narratives side by side.
WATCH FOR	Any path where the system files, notifies a regulator, or closes a finding without a human. That is an automatic fail on this brief, and rightly.

`Bedrock KB` `Guardrails` `Step Functions` `AgentCore Identity` `DynamoDB`

UC-09 Field service quotes that hold their margin

Commercial operations

> *Price realisation versus list — how much discount leaks.*
>
> A KPI clients ask for, and cannot produce

> A KPI clients ask for and cannot produce

Industrial services quoting depends on a handful of experienced estimators. Turnaround is slow, the knowledge is in their heads, and margin leaks quietly through discounts nobody aggregates. When an estimator retires, the capability goes with them.

Build the quoting crew: turn an inbound request — email, spreadsheet, scanned drawing — into a structured job spec; price labour, equipment and travel from historical jobs; flag anything below the margin floor *with the reason*; and check crew and equipment feasibility before a human presses send.

AGENTS	`orchestrator` → `rfq-parser`, `estimator`, `margin-guard`, `feasibility-checker`, `critic` + **human send approval**
DATA SHAPE	Twelve inbound requests in mixed formats, 2,000 historical jobs with labour, equipment, travel and realised margin
CUT LINE	One service line, email and spreadsheet inputs only. Skip the scheduler; keep the margin guard — it is the part a CFO cares about.
DEMO MOMENT	A messy email in; a priced quote out in forty seconds, with one line flagged *"3 points below floor — travel to this site has run over on the last four jobs"*.
WATCH FOR	A quote with no comparable behind it. Every price cites the historical jobs it came from.

`Textract` `Bedrock KB` `DynamoDB` `Step Functions` `AgentCore Browser`

UC-10 The debrief that writes the playbook

Internal capability

> *One simple rule we should introduce at the end of every engagement: the team writes up learnings into reusable pieces. Nobody wants to do documentation.*
>
> Engineering leadership, September 2026

This is our own constraint, and it is the reason the other nine briefs matter less than they look. We cannot deliver the next four engagements the way we delivered the last one — not because the work is hard, but because the learning does not survive the engagement. People will talk for an hour. They will not write for an hour.

So invert it. Build a facilitator agent that runs a structured debrief by asking good reflection questions, a second agent that turns the unstructured conversation into a template or checklist, a third that diffs it against the existing playbook and proposes *only the delta*, and a librarian that files the result where the delivery agents will read it.

AGENTS	`orchestrator` → `facilitator` (asks, does not summarise), `artefact-extractor`, `playbook-differ`, `librarian`, `critic`
DATA SHAPE	One real recorded team debrief — run it live at the hackathon, on the hackathon — plus an existing playbook page to diff against
CUT LINE	Facilitator plus extractor plus differ. The librarian can write to a folder.

DEMO MOMENT	Debrief your own two days. Show the checklist it produced, and the three lines it proposed adding to the playbook.
WATCH FOR	Producing a summary. A summary is not a reusable artefact — a checklist someone follows next time is. The test is whether another team could use the output tomorrow.

`Transcribe` `Bedrock` `Bedrock KB` `AgentCore Memory` `S3 versioning`

04

Delivery guidelines

Same repo shape for all ten, so a judge can navigate any team's work in two minutes and so the good ideas survive past Friday.

Repository layout

```
uc-07-forecast-scoping/
├── README.md          what it does · who for · run it in 5 commands
├── ARCHITECTURE.md    the diagram, the data flow, why each service
├── AGENTS.md          one row per agent — graded hardest, see below
├── EVALS.md           the fixed input set, the scores, what still fails
├── DEMO.md            the 3-minute script, literally, with timings
├── COST.md            cost per run, and at 100 runs/day
├── DECISIONS.md       5 lines per decision: what, why, what we rejected
├── HANDOFF.md         what is real, what is faked, what breaks first
├── agents/            one file per agent, prompt + tool bindings
├── tools/             deterministic code the agents call
├── evals/             fixed inputs + expected outputs + the runner
├── infra/             CDK or SAM — it has to deploy from clean
└── demo/              the data you will demo on
```

AGENTS.MD — THE ONE THAT DECIDES YOUR DOCUMENTATION SCORE

One table, one row per agent, these seven columns: **name · single responsibility · input schema · output schema · tools it may call · guardrail · failure mode**. If you cannot fill the row, the agent is not designed yet — and writing this table on hour three is the cheapest architecture review you will get all week. Teams that write it at the end always discover two agents doing the same job.

The two days

DAY 1 · 09:00–10:30

Pick the brief. Name the executive who wants it. Write `AGENTS.md` — the table, before any code. Draw the orchestration on a whiteboard and photograph it into `ARCHITECTURE.md`.

DAY 1 · 10:30–12:00

Build the eval set *first*: ten fixed inputs, expected outputs, including one input designed to fail and one that should be rejected. This is the single highest-leverage ninety minutes of the two days.

DAY 1 · 12:00–16:00

Tools before agents. Write the deterministic functions and unit-test them. Agents built on untested tools debug very badly, because you cannot tell whether the model or the function was wrong.

DAY 1 · 16:00–18:00

One specialist agent, end to end, with the run ledger writing. Prove the skeleton on one path before adding the second agent.

DAY 2 · 09:00–12:00

Remaining specialists, then the orchestrator, then the critic. Run the eval set after each addition — a score that drops tells you which agent did it.

DAY 2 · 12:00–14:00

Human-in-the-loop step. Guardrails on. Budget ceiling enforced. These three take about an hour together and they are what make it showable to a client.

DAY 2 · 14:00–16:00

Freeze the code. Write the docs. Record the eval scores as they actually are, failures included.

DAY 2 · 16:00–17:00

Rehearse the demo twice, on the clock, from a cold start. A demo that needs a warm cache is a demo that fails on stage.

> **THE 14:00 FREEZE ON DAY TWO IS REAL**
>
> Every hackathon produces at least one team with something genuinely good and no documentation, and their work is unusable a week later — which makes those two days a total loss regardless of how the demo went. Freeze, then write. If the freeze costs you a feature, it was the right feature to lose.

Documenting an agent well

A well-documented agent is one a different engineer can modify without reading your prompt. That takes four things, and prose is not one of them:

- **A schema on both sides.** Typed input, typed output, validated at the boundary. An agent whose output shape is "whatever the model said" cannot be composed with anything.
- **An explicit tool list.** What it may call, and — worth stating — what it deliberately may not.
- **A named failure mode.** What this agent does when it cannot do its job. Returning a confident guess is the wrong answer; say what the right one is.

- **One worked example in the repo.** A real input and its real output, committed. It doubles as a regression test and it is the fastest way for anyone to understand what the agent is for.

05

Definition of done

Scored out of 100. Note where the weight sits: how well it orchestrates and how much it can be trusted together outweigh whether the demo dazzles.

Client value A named portfolio-company executive would pay for this, and you can say what it replaces	25
Orchestration quality Real roles with real contracts; the orchestrator plans and does not do domain work	20
Trustworthiness Citations throughout, a critic that actually rejects, guardrails, a human in the loop before any write	20
Documentation The eight files. `AGENTS.md` complete. Another team could pick this up on Monday	15
Works on unseen input We will hand you an input you have not seen. Degrading honestly beats failing loudly	10
Cost and path to production Cost per run measured, not guessed; an honest account of what breaks at scale	10

Hard gates

Below any of these, the score does not matter:

- It deploys from a clean account with the documented commands.
- The eval set runs and its results are written down — *including* the failures. A perfect `EVALS.md` is treated as a missing one.
- No autonomous write to any system of record.
- No real client data anywhere in the repo, the logs or the prompts.
- `AGENTS.md` exists and every row is complete.

> WHAT WE ARE ACTUALLY SHOPPING FOR
>
> Two or three of these become real engagements. The ones that get picked will not be the flashiest; they will be the ones where a judge could read `HANDOFF.md` and see exactly what week three looks like. Write that file for a colleague who is about to be

> exactly what week three looks like. Write that file for a colleague who is about to be given your prototype and no context.

06

Default stack

Use these unless you have a reason not to, and put the reason in `DECISIONS.md`. Nobody should spend hackathon hours choosing a vector store.

JOB	USE	WHY THIS ONE
Models	**Amazon Bedrock**	One endpoint, one IAM story, guardrails attach natively
Agent framework	**Strands Agents**	Multi-agent patterns are first class; an agent-as-tool is a decorator, not a framework fight
Agent runtime	**Bedrock AgentCore**	Session isolation, Memory, Identity, Observability and a Code Interpreter without building any of it
Retrieval	**Bedrock Knowledge Bases**	S3 in, chunking and embedding handled, citations come back with the chunk
Vectors	**OpenSearch Serverless**	The Knowledge Bases default. Do not litigate it in two days
Deterministic flow	**Step Functions**	Where a step order is known, encode it. Also the cleanest human-approval primitive you have
Run ledger	**DynamoDB**	One table, `run_id` partition key. Claims, evidence, verdicts, tokens
Documents	**Textract**	Tables and forms survive extraction, which is the whole problem with PDFs
Speech	**Transcribe**	Diarisation and custom vocabulary — both load-bearing for UC-02
Tabular at volume	**Athena over S3**	No cluster to stand up; an agent can hold a SQL tool against it
Arithmetic	**Code Interpreter**	Every number a model states unaided is a number you cannot defend
Safety	**Bedrock Guardrails**	PII and topic filters, one config, no code

Where the orchestration should *not* be an agent

A workflow that always runs the same steps in the same order is code, not a reasoning problem. Put it in Step Functions and call the agent only at the step that actually needs judgement. Your demo will be faster, cheaper, and you will be able to explain it.

An orchestrator that plans is worth its cost when the next step genuinely depends on what was just found. When the order is known in advance — extract, then classify, then score, then write — that is a state machine, and a state machine is cheaper, faster, debuggable and does not wander. Most of these ten briefs want a Step Functions spine with agents at the nodes, and exactly one planning agent where the path is genuinely open. Saying which is which in `ARCHITECTURE.md` earns marks; making everything an agent loses them.

07

Traps

All of these have been shipped, most of them by people who knew better, several on this account.

TRAP	HOW IT SHOWS UP	WHAT TO DO INSTEAD
The mega-prompt	One agent, four thousand tokens of instructions, no roles. Works on the demo input and nothing else	Split by responsibility. If you cannot name an agent's one job in six words, it is two agents
Agent sprawl	Nine agents by lunch on day two, none individually tested, and no idea which one is wrong	Add the next agent only when the eval score for the current set is understood
Chat as the interface	A chat box in front of a task with four known inputs	A form. Agentic refers to what happens behind the screen, not to a conversation on it
Demo-only data	Flawless on the one PDF you tuned against; collapses on the second	Ten inputs in the eval set from hour two, at least three of them awkward
No critic	Confident, fluent, unverifiable output. The most expensive failure here, because it looks like success	Build the critic third, not last, and demo it catching something
A loop with no floor	Reject, re-plan, reject. No error and no crash — just a run that never ends and a bill that does not stop. Found by a reviewer reading our own diagram as a state machine	Retry ceiling in the ledger, a guard on the rejected edge, a rejected-record store at the end of it
Two sources of truth	The summary card and the detail table disagree, because each computes the total its own way	One function, called twice. This exact bug has shipped four times on this account
Autonomous writes	"It updates the ERP automatically." No client will let this near production, so the demo is unbuyable	Human approval before every write, from the first commit

Docs after the demo	Day two, 17:30, a README that says `npm start`	The 14:00 freeze. It is not negotiable and it is not a suggestion
Scraping past an API	A browser agent clicking through a portal that has a documented REST endpoint	Use the API. Reserve the browser tool for systems that genuinely have no other door
Solving our problem, not theirs	A tool that makes consultants faster, pitched as client value	Legitimate — UC-10 is exactly this — but say so. Do not dress internal efficiency up as a client outcome

---